

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_1d39e81c-851\agent-ac1ed67c2c4f5846b.jsonl`  
- SHA-256: `5c276b1febfc84999517507ce7cf9f87565888eaa2ddaa4eb1aeed2db26ebab5`  
- Source modified: `2026-06-18T10:22:13+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_1d39e81c-851`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T10:20:58.212Z)

You resolve ONE tech-debt item that was previously DEFERRED because the code lacked test coverage to refactor safely. You work in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine. The Python env HAS pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

THE DISCIPLINE (do in this exact order):

1. SETUP: git fetch origin && git checkout -B <BRANCH> origin/master (base MUST be origin/master).
Verify HEAD == origin/master and branch == <BRANCH>.
2. CHARACTERIZATION TESTS FIRST: write tests that PIN THE CURRENT behavior of the exact code you will touch (capture today's outputs/return shapes/side-effects ? bugs and all; do NOT "fix" anything).
Run them against the UNCHANGED code and CONFIRM THEY PASS. If you cannot pin the behavior deterministically (e.g. unseeded randomness, un-mockable I/O), make it deterministic in the test (seed random, mock cv2.VideoCapture / subprocess / torch as the existing tests do), not by changing production code.
3. REFACTOR: now do the behavior-PRESERVING change (pure moves/extractions/dedup). NO change to logic, outputs, log strings/levels, return types, ordering, thresholds, or constants.
4. RE-VERIFY: the SAME characterization tests must still pass unchanged.

HARD RULES:

- If you discover the item actually requires CHANGING behavior or is a redesign (not a pure move), STOP ? do NOT force it. Report status='held' with the reason and what you learned.
- Edit ONLY the files named in your task (+ your new test file). Stage EXPLICIT paths (never -A/./-u).
- Match surrounding style. Never weaken a test or lower the coverage floor to go green.

GATE (binding ? capture pass/skip counts + coverage %):

```
python run_all_tests.py  
python -m ruff check .  
python -m mypy backend training  
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70  
ONLY IF ALL FOUR GREEN and coverage did NOT drop vs base (~80.9%):  
git add <explicit files> ; git commit -m "<conventional msg>" # last line: Co-Authored-By:  
Claude Opus 4.8 <noreply@anthropic.com>  
git push -u origin <BRANCH> ; gh pr create --base master --head <BRANCH> --title "... " --body  
"... " # NO merge
```

ELSE report status='gate-failed' (or 'held') with the failing output. Do NOT push a red branch.

Return the structured object: slug, branch, status ('pr-opened' | 'held' | 'gate-failed' | 'error'),
pr_url ("" + reason if none), char_tests (what you added + that they passed pre-refactor),
diffstat,
suite, ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR ITEM: base-resolve-provider-dedup (branch refactor/base-resolve-provider-dedup) ===
ITEM: gemini.py and yolo_hybrid.py (both just refactored on master) now each have an IDENTICAL
private _resolve_provider(self) preamble (sport/provider/api-key/registry ? provider | None,
signalling the
early return-[] cases). Lift the now-identical body to the shared base class
backend/pipeline/segmenters/
base.py (e.g. a protected _resolve_provider on SegmenterABC) and have gemini + yolo_hybrid call
the base
method. EXCLUDE trajectory_hybrid (its variant differs ? skip_ai branch; do NOT touch it).
CHAR TESTS FIRST: the early-return branches (unknown sport / missing API key / unknown provider)
are
under-tested. In tests/test_gemini.py and tests/test_yolo_hybrid.py (or a new
tests/test_segmenter_base.py)
add tests pinning each early return [] for BOTH segmenters (no API key, unknown provider,
unknown sport),
passing on UNCHANGED code. THEN dedup. Confirm gemini/yolo still return the SAME values and the
bare-list
contract holds. Only lift if the two bodies are byte-identical; if they diverge, report
status='held'.

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_1d39e81c-851-2 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T10:21:00.834Z)

I'll start by setting up the worktree, then write characterization tests before refactoring.

3. user (2026-06-18T10:21:04.237Z)

branch 'refactor/base-resolve-provider-dedup' set up to track 'origin/master'.
Switched to a new branch 'refactor/base-resolve-provider-dedup'

4. assistant (2026-06-18T10:21:06.416Z)

Branch is set up. Let me verify HEAD matches origin/master and read the target files.

5. user (2026-06-18T10:21:08.862Z)

```
=== HEAD ===
e67add172f5b620025f31489fde2fb9ff45465ac
=== origin/master ===
e67add172f5b620025f31489fde2fb9ff45465ac
=== branch ===
refactor/base-resolve-provider-dedup
```

6. assistant (2026-06-18T10:21:10.852Z)

HEAD == origin/master and branch is correct. Now let me read the target files.

7. user (2026-06-18T10:21:11.823Z)

```
1 import os
2 import time
```

```

3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple, Callable
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    def _parse_time(val) -> float:
27        """Converts a timestamp value to total float seconds.
28        Handles float/int pass-through, plain numeric strings, and
29        colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
30        when unexpected formats are encountered."""
31        if isinstance(val, (int, float)):
32            return float(val)
33        if isinstance(val, str):
34            val = val.strip()
35            if ":" in val:
36                # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
37                # Convert it, but warn so the issue is visible in logs.
38                parts = val.split(":")
39                parts.reverse()
40                total = 0.0
41                for i, p in enumerate(parts):
42                    try:
43                        total += float(p) * (60 ** i)
44                    except ValueError:
45                        pass
46                logger.warning(f"Timestamp '{val}' was in colon-separated format
47(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
48                return total
49            try:
50                return float(val)
51            except ValueError:
52                logger.warning(f"Could not parse timestamp string '{val}' as a number.
53Defaulting to 0.0.")
54                return 0.0
55
56
57    class GeminiPlaySegmenter(BasePlaySegmenter):
58        @property
59        def name(self) -> str:
60            return "gemini"
61
62        @property
63        def description(self) -> str:
64            return "High-precision AI-powered semantic play segment detection using Google

```

```

Gemini Multimodal API."
65
66     def _resolve_provider(self, model_name: str):
67         """Resolve the sport profile, provider, and API key (the process_video
preamble).
68
69         Returns (sport_profile, provider) on success, or None to signal any of the
70         early-return cases (unknown sport, missing API key, unknown provider) ? the
71         caller returns [] for None.
72         """
73         # Get sport profile config
74         sport_name = self.config.get("sport", "badminton")
75         try:
76             sport_profile = sport_registry.get(sport_name)
77         except KeyError as e:
78             logger.error(str(e))
79             return None
80
81         # Get AI provider and model config
82         idx_cfg = self.config.get("indexing", {})
83         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
84
85         # Get appropriate API key based on the provider
86         api_key_env_var = f"{provider_name.upper()}_API_KEY"
87         api_key = os.environ.get(api_key_env_var)
88         if not api_key:
89             logger.error(
90                 f"{api_key_env_var} environment variable is not set! "
91                 f"To run the {provider_name} segmenter, set your API key. "
92                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
93             )
94             return None
95
96         try:
97             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
98         except KeyError as e:
99             logger.error(str(e))
100             return None
101
102         return sport_profile, provider
103
104     def _maybe_trim_video(self, video_id: str, video_path: str) -> Tuple[str,
Callable[[], None]]:
105         """Optionally trim the video to indexing.debug_duration seconds (debug feature).
106
107         Returns (video_to_upload, cleanup_temp_files) where cleanup_temp_files removes
any
108         temp trimmed file created here.
109         """
110         # A: Check for debug_duration to trim the video first
111         debug_duration = self.config.get("indexing", {}).get("debug_duration")
112         video_to_upload = video_path
113         is_trimmed = False
114         temp_trimmed_path = None
115
116         if debug_duration:
117             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
118             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
119             os.makedirs("output", exist_ok=True)
120             if os.path.exists(temp_trimmed_path):
121                 try:
122                     os.remove(temp_trimmed_path)

```

```

123         except Exception:
124             pass
125
126         cmd = [
127             "ffmpeg", "-y",
128             "-ss", "0",
129             "-t", str(debug_duration),
130             "-i", video_path,
131             "-c", "copy",
132             temp_trimmed_path
133         ]
134         logger.debug(f"Running command: {' '.join(cmd)}")
135         try:
136             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
137             logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
138             video_to_upload = temp_trimmed_path
139             is_trimmed = True
140         except Exception as e:
141             logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
142
143         # Helper to clean up local temp files
144         def cleanup_temp_files():
145             if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
146                 try:
147                     os.remove(temp_trimmed_path)
148                 except Exception:
149                     pass
150
151         return video_to_upload, cleanup_temp_files
152
153     def _process_chunked(
154         self,
155         video_id: str,
156         video_to_upload: str,
157         sport_profile,
158         provider,
159         video_duration: float,
160         chunk_duration: float,
161         chunk_overlap: float,
162         ai_scale_width: int,
163         chunks_dir: str,
164         cleanup_temp_files: Callable[[[]], None],
165     ) -> List[Dict[str, Any]]:
166         """--- CHUNKED PROCESSING --- (long videos): split into overlapping chunks,
167         analyze them in parallel, offset timestamps back to full-video coordinates."""
168         ai_segments: List[Dict[str, Any]] = []
169         step = chunk_duration - chunk_overlap
170         chunk_starts = []
171         t = 0.0
172         while t < video_duration:
173             chunk_starts.append(t)
174             t += step
175         num_chunks = len(chunk_starts)
176         logger.info(
177             f"Video is {video_duration:.1f}s ? splitting into {num_chunks} overlapping
chunks "
178             f"({chunk_duration}s each, {chunk_overlap}s overlap)."
179         )
180
181         os.makedirs(chunks_dir, exist_ok=True)
182
183         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list, dict]:

```

```

184         chunk_end = min(chunk_start + chunk_duration, video_duration)
185         actual_chunk_dur = chunk_end - chunk_start
186         chunk_label = f"Chunk {ci+1}/{num_chunks}"
187         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
188
189         # Extract chunk with ffmpeg
190         logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
191         success = extract_video_chunk(
192             video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
193             reencode=ai_scale_width > 0,
194             scale_width=ai_scale_width if ai_scale_width > 0 else None,
195         )
196         if not success:
197             logger.error(f"Failed to extract {chunk_label}. Skipping.")
198             return chunk_label, [], {}
199
200         # Measure chunk size
201         try:
202             chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
203         except Exception:
204             chunk_size_mb = 0.0
205
206         # Get prompt from sport profile
207         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
208
209         # Process this chunk through the provider
210         chunk_segments, metrics = provider.analyze_video(
211             chunk_path, prompt, chunk_duration=actual_chunk_dur, label=chunk_label
212         )
213         metrics["file_size_mb"] = chunk_size_mb
214         metrics["segments_found"] = len(chunk_segments)
215
216         # Offset timestamps back to full-video coordinates
217         for segment in chunk_segments:
218             if "start_time" in segment:
219                 try:
220                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
221                 except (ValueError, TypeError):
222                     pass
223             if "end_time" in segment:
224                 try:
225                     segment["end_time"] = float(segment["end_time"]) + chunk_start
226                 except (ValueError, TypeError):
227                     pass
228
229         logger.info(f"{chunk_label} returned {len(chunk_segments)} play segments.")
230
231         # Clean up chunk file
232         try:
233             os.remove(chunk_path)
234         except Exception:
235             pass
236
237         return chunk_label, chunk_segments, metrics
238
239     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same knob
240     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
241     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns a
242     # misleading "credits depleted" 429 under 5 simultaneous uploads.
243     all_metrics = {}
244     max_workers = max(1, int(self.config.get("indexing", {}).get("ai_max_workers",
5)))
245     logger.info("Starting parallel processing with up to %d concurrent workers...",

```

```

max_workers)
246         t_chunking_start = time.time()
247         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
248             futures = [executor.submit(process_single_chunk, ci, start) for ci, start in
enumerate(chunk_starts)]
249             for future in concurrent.futures.as_completed(futures):
250                 label, segments, metrics = future.result()
251                 ai_segments.extend(segments)
252                 if metrics:
253                     all_metrics[label] = metrics
254
255             # Oversize refusals must be LOUD: each one is a time range with NO analysis,
256             # so "0 segments / success" here would be a silent failure, not "no rallies".
257             oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
258             if oversize_skips:
259                 logger.error(
260                     f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload cap
and were "
261                     f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges.
"
262                     f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
263                     f"downscaled before upload."
264                 )
265             run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
266
267             # Print Summary Log
268             summary_lines = [
269                 "",
270                 "=" * 60,
271                 "                GEMINI CHUNKING SUMMARY",
272                 "=" * 60,
273                 f" Total time: {time.time() - t_chunking_start:.1f}s",
274                 f" Total chunks: {num_chunks}",
275                 f" Total play segments found: {len(ai_segments)}",
276                 "-" * 60,
277                 f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments':<8}",
278                 "-" * 60,
279             ]
280             for i in range(num_chunks):
281                 lbl = f"Chunk {i+1}/{num_chunks}"
282                 m = all_metrics.get(lbl, {})
283                 sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
284                 upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
285                 prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
286                 gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
287                 seg = m.get('segments_found', 0) if m else 0
288                 summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} | {gen:<8}
| {seg:<8}")
289             summary_lines.append("=" * 60)
290             logger.info("\n".join(summary_lines))
291
292             # Clean up local temp files
293             cleanup_temp_files()
294             try:
295                 os.rmdir(chunks_dir)
296             except Exception:
297                 pass
298
299             logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
300
301             return ai_segments

```

```

302
303     def _process_single(
304         self,
305         video_id: str,
306         video_to_upload: str,
307         sport_profile,
308         provider,
309         video_duration: float,
310         cleanup_temp_files: Callable[[], None],
311     ) -> List[Dict[str, Any]]:
312         """--- SINGLE-FILE PROCESSING (short videos) --- analyze the whole video in one
313         call."""
314         try:
315             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
316         except Exception:
317             file_sz = 0.0
318
319         # Get prompt from sport profile
320         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
321         ai_segments, metrics = provider.analyze_video(
322             video_to_upload, prompt, chunk_duration=video_duration, label="SingleFile"
323         )
324         if metrics.get("oversize_skipped"):
325             logger.error(
326                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and was "
327                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
328                 rallies'."
329             )
330             run_signals.incr(video_id, "oversize_clips_skipped", 1)
331         summary_lines = [
332             "",
333             "=" * 50,
334             "    GEMINI SINGLE FILE SUMMARY",
335             "=" * 50,
336             f" File Size: {file_sz:.1f} MB",
337             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
338             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
339             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
340             f" Play segments found: {len(ai_segments)}",
341             "=" * 50,
342         ]
343         logger.info("\n".join(summary_lines))
344         cleanup_temp_files()
345
346         return ai_segments
347
348     def _validate_and_dedupe(self, ai_segments: List[Dict[str, Any]], video_duration:
349     float) -> List[Dict[str, Any]]:
350         """Parse timestamps, validate/clamp candidates, and resolve overlaps (keep
351         longer)."""
352         logger.info(f"Processing {len(ai_segments)} play segments through
353         validation...")
354
355         # 7. Parse timestamps and build candidate segment list
356         idx_cfg = self.config.get("indexing", {})
357         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
358         max_segment_duration = 90.0 # Singles play segments almost never exceed this
359
360         candidates = []
361         for idx, segment in enumerate(ai_segments):
362             start = _parse_time(segment.get("start_time", 0.0))
363             end = _parse_time(segment.get("end_time", 0.0))
364             if start >= end:
365                 logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
366                 end_time ({end:.2f}s)")

```



```

361         continue
362     candidates.append({
363         "idx": idx,
364         "start": start,
365         "end": end,
366         "raw": segment
367     })
368
369     # 8. Post-hoc validation pass
370     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
371     validated = []
372     rejected = []
373
374     for c in candidates:
375         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
376
377         # A: Reject segments that start beyond the video
378         if video_duration > 0 and c["start"] >= video_duration:
379             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
380             continue
381
382         # B: Clamp end_time to video duration
383         if video_duration > 0 and c["end"] > video_duration:
384             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
385             c["end"] = video_duration
386
387         dur = c["end"] - c["start"]
388
389         # C: Reject segments shorter than min_segment_duration
390         if dur < min_segment_duration:
391             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
392             continue
393
394         # D: Flag suspiciously long segments
395         if dur > max_segment_duration:
396             logger.warning(
397                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
398                 f"This may be a detection error. Keeping but flagging."
399             )
400
401         validated.append(c)
402
403     # E: Resolve overlapping segments (keep the longer one)
404     if len(validated) > 1:
405         # Sort by start time
406         validated.sort(key=lambda r: r["start"])
407         non_overlapping = [validated[0]]
408         for curr in validated[1:]:
409             prev = non_overlapping[-1]
410             # Check if current segment overlaps with the previous one
411             if curr["start"] < prev["end"]:
412                 prev_dur = prev["end"] - prev["start"]
413                 curr_dur = curr["end"] - curr["start"]
414                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
415                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
416                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
417                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept

```

```

the longer segment"))
418             if kept == curr:
419                 non_overlapping[-1] = curr # Replace previous with current
420             else:
421                 non_overlapping.append(curr)
422             validated = non_overlapping
423
424         # Log validation summary
425         if rejected:
426             reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
427             logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
428             logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
429
430             stitch_cfg = self.config.get("stitching", {})
431             begin_pad = stitch_cfg.get("begin_padding", 0.5)
432             end_pad = stitch_cfg.get("end_padding", 0.5)
433
434             total_highlight_dur = 0.0
435             for c in validated:
436                 total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
437
438             mins = int(total_highlight_dur // 60)
439             secs = int(total_highlight_dur % 60)
440             logger.info(
441                 f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
442                 f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
443             )
444
445             return validated
446
447         def _populate_db(self, video_id: str, validated: List[Dict[str, Any]], model_name:
str) -> List[Dict[str, Any]]:
448             """9. Populate SQLite DB with validated play segments; returns segments_data."""
449             segments_data = []
450             for c in validated:
451                 start = c["start"]
452                 end = c["end"]
453                 segment = c["raw"]
454                 dur = end - start
455
456                 # Extract shot_count from Gemini or compute estimate if not present
457                 shot_count = segment.get("shuttle_exchange_count")
458                 if shot_count is None:
459                     shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
460                 else:
461                     try:
462                         shot_count = int(shot_count)
463                     except Exception:
464                         shot_count = max(3, int(dur / 0.8))
465
466                 # Only store metrics we actually know ? no fabricated data
467                 metadata = {
468                     "shot_count": shot_count,
469                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
470                     "detector": model_name
471                 }
472
473                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
474

```

```

475         if segment_id:
476             segments_data.append({
477                 "id": segment_id,
478                 "start_time": start,
479                 "end_time": end,
480                 "duration": dur,
481                 "metadata": metadata
482             })
483
484     return segments_data
485
486     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
487         """
488         Processes a video to detect play segments using Google Gemini Multimodal Video
API,
489         populates SQLite, and indexes detailed events.
490         """
491         model_name = self.config.get("ai_model", self.config.get("indexing",
{})).get("ai_model", "gemini-2.5-flash"))
492
493         # Resolve sport profile + provider (handles the unknown-sport / missing-key /
unknown-provider early returns)
494         resolved = self._resolve_provider(model_name)
495         if resolved is None:
496             return []
497         sport_profile, provider = resolved
498
499         # Optionally trim the video first (debug feature); cleanup removes any temp
file.
500         video_to_upload, cleanup_temp_files = self._maybe_trim_video(video_id,
video_path)
501
502         # 1. Determine actual video duration
503         video_duration = get_video_duration(video_to_upload)
504         if video_duration > 0:
505             logger.info(f"Detected video duration: {video_duration:.1f}s")
506         else:
507             logger.warning("Could not detect video duration. Prompt will not include
duration context.")
508
509         # 2. Decide whether to use chunked processing
510         idx_cfg = self.config.get("indexing", {})
511         chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
512         chunk_overlap = idx_cfg.get("chunk_overlap", 30)      # 30 seconds overlap
513         use_chunking = video_duration > 0 and video_duration > chunk_duration
514         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
515         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
516         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
517         ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
518
519         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
520         # video keeps its own dir, so cleanup here never removes another run's chunks.
521         chunks_dir = _chunk_workdir(video_id)
522
523         if use_chunking:
524             ai_segments = self._process_chunked(
525                 video_id, video_to_upload, sport_profile, provider, video_duration,
526                 chunk_duration, chunk_overlap, ai_scale_width, chunks_dir,
cleanup_temp_files,
527             )
528         else:
529             ai_segments = self._process_single(
530                 video_id, video_to_upload, sport_profile, provider, video_duration,

```

```

cleanup_temp_files,
531         )
532
533         if not ai_segments:
534             return []
535
536         # Populate SQLite DB with parsed play segments
537         validated = self._validate_and_dedupe(ai_segments, video_duration)
538
539         # 9. Populate SQLite DB with validated play segments
540         segments_data = self._populate_db(video_id, validated, model_name)
541
542         return segments_data
543
544     # Register the segmenter
545     segmenter_registry.register(GeminiPlaySegmenter)
546

```

8. user (2026-06-18T10:21:12.502Z)

```

1     """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2     (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4     Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5     (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6     can be consumed by the future fusion segmenter; this segmenter just orchestrates
7     chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9     Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10    service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11    detection uses torchvision's permissively-licensed (BSD) detectors and the
12    association/tracking that YOLO.track() used to bundle in is provided by
13    supervision's MIT-licensed ByteTrack.
14
15    The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16    DB `detector` tags, even though no YOLO is involved anymore.
17    """
18    import os
19    import time
20    import logging
21    import concurrent.futures
22    from typing import List, Dict, Any, Tuple
23
24    from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25    from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26    from backend.utils.video import get_video_duration, extract_video_chunk
27    from backend.sports import sport_registry
28    from backend.providers import provider_registry
29
30    logger = logging.getLogger(__name__)
31
32
33    def _fmt_ts(seconds: float) -> str:
34        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
35        seconds = max(0, int(seconds))
36        h, rem = divmod(seconds, 3600)
37        m, s = divmod(rem, 60)
38        return f"{h:02d}:{m:02d}:{s:02d}"
39
40
41    class HybridYoloSegmenter(BasePlaySegmenter):
42        def __init__(self, db, config):
43            super().__init__(db, config)
44            # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
45    producer

```

```

45         # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
46         self.person = PersonDetector(config)
47
48         @property
49         def name(self) -> str:
50             return "yolo_hybrid"
51
52         @property
53         def description(self) -> str:
54             return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
55                     "candidate filtering, then an AI provider for high-precision semantic
56 boundaries.")
57
58         def _resolve_provider(self, video_path: str):
59             """Phase 0: resolve sport profile + AI provider (with API key).
60
61             Returns the configured provider instance, or ``None`` to signal that
62             ``process_video`` should early-return ``[]`` (bad sport, missing API key,
63             or unknown provider). Behaviour is identical to the inlined version.
64             """
65             # Get sport profile config
66             sport_name = self.config.get("sport", "badminton")
67             try:
68                 sport_registry.get(sport_name)
69             except KeyError as e:
70                 logger.error(str(e))
71                 return None
72
73             # Get AI provider and model config
74             idx_cfg = self.config.get("indexing", {})
75             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
76 "gemini"))
77             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
78 "gemini-2.5-flash"))
79
80             # Get appropriate API key based on the provider
81             api_key_env_var = f"{provider_name.upper()}_API_KEY"
82             api_key = os.environ.get(api_key_env_var)
83             if not api_key:
84                 logger.error(
85                     f"{api_key_env_var} environment variable is not set! "
86                     f"To run the {provider_name} segmenter, set your API key. "
87                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
88                 return None
89
90             try:
91                 provider = provider_registry.get(provider_name, api_key=api_key,
92 model_name=model_name)
93             except KeyError as e:
94                 logger.error(str(e))
95                 return None
96
97             logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
98 {video_path}")
99             return provider
100
101         def _collect_candidate_windows(
102             self, video_id: str, video_path: str, video_duration: float, idx_cfg: Dict[str,
103 Any],
104             chunk_duration: float, chunk_overlap: float, velocity_thresh: float, merge_gap:
105 float,
106             frame_skip: int, chunks_dir: str,
107 ) -> List[Dict[str, Any]]:
108             """Phase 2: chunk the video, extract per-chunk action windows (pipelined or

```

```

103         sequential), then dedup-merge overlapping windows across chunks. Verbatim
move. """
104         step = chunk_duration - chunk_overlap
105         chunk_starts = []
106         t = 0.0
107         while t < video_duration:
108             chunk_starts.append(t)
109             t += step
110
111         num_chunks = len(chunk_starts)
112         logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
113
114         all_candidate_windows = []
115
116         t_start = time.time()
117         prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
118
119         if num_chunks > 1 and prefetch_workers > 0:
120             # --- PIPELINED PROCESSING ---
121             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
122             # extraction is pure I/O. We pre-extract upcoming chunks in background
123             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
124             logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
125
126             extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
127
128             def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
129                 chunk_end = min(cs + chunk_duration, video_duration)
130                 actual_dur = chunk_end - cs
131                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
132                 success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
133                 return (ci, cs, chunk_path, success)
134
135             # Submit all extractions upfront ? they'll queue and run as workers free up
136             extract_futures = [
137                 extract_executor.submit(_extract_chunk, ci, cs)
138                 for ci, cs in enumerate(chunk_starts)
139             ]
140
141             # Process results in order: wait for extraction, then run inference on GPU
142             for future in extract_futures:
143                 ci, chunk_start, chunk_path, success = future.result()
144                 chunk_end = min(chunk_start + chunk_duration, video_duration)
145                 logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
146
147                 if not success:
148                     logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
149                     continue
150
151                 windows = self.person.extract_action_windows_from_chunk(
152                     chunk_path, chunk_start, velocity_thresh, merge_gap,
153                     frame_skip=frame_skip, chunk_index=ci
154                 )
155                 logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
156                 all_candidate_windows.extend(windows)
157
158             try:
159                 os.remove(chunk_path)
160             except Exception:
161                 pass

```

```

162         extract_executor.shutdown(wait=False)
163     else:
164         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
165         for ci, chunk_start in enumerate(chunk_starts):
166             chunk_end = min(chunk_start + chunk_duration, video_duration)
167             actual_dur = chunk_end - chunk_start
168             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
169
170             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
171 {chunk_end:.1f}s)...")
172             success = extract_video_chunk(video_path, chunk_start, actual_dur,
173 chunk_path)
174             if not success:
175                 continue
176
177             windows = self.person.extract_action_windows_from_chunk(
178                 chunk_path, chunk_start, velocity_thresh, merge_gap,
179                 frame_skip=frame_skip, chunk_index=ci
180             )
181             logger.info(f"Found {len(windows)} candidate action windows in chunk
182 {ci+1}.")
183             all_candidate_windows.extend(windows)
184
185             try:
186                 os.remove(chunk_path)
187             except Exception:
188                 pass
189
190 # Deduplicate overlapping candidate windows across chunks (chunks overlap by
191 # chunk_overlap, so the same rally can surface in two adjacent chunks).
192 def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
193     fa, fb = a["active_frame_count"], b["active_frame_count"]
194     total = fa + fb or 1
195     return {
196         "start": a["start"],
197         "end": max(a["end"], b["end"]),
198         "active_frame_count": fa + fb,
199         "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
200         # Frame-weighted mean so longer sub-windows dominate proportionally.
201         "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
202 total,
203         "track_id_count": max(a["track_id_count"], b["track_id_count"]),
204         "chunk_index": a["chunk_index"],
205     }
206
207 if all_candidate_windows:
208     all_candidate_windows.sort(key=lambda w: w["start"])
209     merged_windows = [all_candidate_windows[0]]
210     for current in all_candidate_windows[1:]:
211         last = merged_windows[-1]
212         if current["start"] <= last["end"]: # Overlap
213             merged_windows[-1] = _merge_stats(last, current)
214         else:
215             merged_windows.append(current)
216     all_candidate_windows = merged_windows
217
218 logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
219 logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
220
221 return all_candidate_windows
222
223 def _persist_candidates(
224     self, video_id: str, all_candidate_windows: List[Dict[str, Any]],
225     detection_params: Dict[str, Any], log_candidates: bool, store_candidates: bool,

```

```

223         ) -> List[Any]:
224         """Phase 2b: per-rally console log + persist raw candidates for the fine-tuning
225         dataset. Returns window-index -> candidate_id (None where not stored). Verbatim
226         move."""
227         # Per-rally console log (final, full-video timestamps) for video cross-
228         verification.
229         if log_candidates:
230             for w in all_candidate_windows:
231                 logger.info(
232                     f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
233                     f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
234                     f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
235                     f"tracks={w['track_id_count']}"
236                 )
237         # Persist raw candidates for the fine-tuning dataset. Map window index ->
238         candidate_id
239         # so confirmed AI segments can be linked back in Phase 4.
240         candidate_ids = [None] * len(all_candidate_windows)
241         if store_candidates:
242             for i, w in enumerate(all_candidate_windows):
243                 stats = {
244                     "peak_velocity": w["peak_velocity"],
245                     "mean_velocity": w["mean_velocity"],
246                     "active_frame_count": w["active_frame_count"],
247                     "track_id_count": w["track_id_count"],
248                 }
249                 # Use peak velocity (capped at 1.0) as a coarse generic confidence
250                 confidence = min(1.0, w["peak_velocity"])
251                 candidate_ids[i] = self.db.add_candidate_segment(
252                     video_id, detector="yolo_hybrid",
253                     start_time=w["start"], end_time=w["end"],
254                     chunk_index=w["chunk_index"], confidence=confidence,
255                     stats=stats, detection_params=detection_params,
256                 )
257         return candidate_ids
258
259     def _run_ai_handoff(
260         self, video_id: str, video_path: str, video_duration: float,
261         all_candidate_windows: List[Dict[str, Any]], candidate_ids: List[Any],
262         provider, sport_profile, provider_name: str, chunk_duration: float,
263         handoff_padding: float, ai_max_workers: int,
264     ) -> List[dict]:
265         """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and ask
266         the provider for semantic boundaries, in parallel. Verbatim move."""
267         gemini_segments = []
268         gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
269         os.makedirs(gemini_dir, exist_ok=True)
270
271         logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
272         {len(all_candidate_windows)} candidate windows...")
273
274         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
275             w_start = max(0, window["start"] - handoff_padding)
276             w_end = min(video_duration, window["end"] + handoff_padding)
277             w_dur = w_end - w_start
278
279             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
280             # Re-encode short handoff clips to avoid keyframe-boundary drift
281             success = extract_video_chunk(video_path, w_start, w_dur, w_path,
282             reencode=True)
283             if not success:

```



```

281         return []
282
283         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
284         segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
285
286         # Coordinate mapping ? shift local timestamps back to full-video time
287         valid_segments = []
288         for segment in segments:
289             try:
290                 segment["start_time"] = float(segment["start_time"]) + w_start
291                 segment["end_time"] = float(segment["end_time"]) + w_start
292                 # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
293                 segment["_candidate_id"] = candidate_ids[wi]
294                 valid_segments.append(segment)
295             except (ValueError, TypeError, KeyError) as e:
296                 logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
297
298             try:
299                 os.remove(w_path)
300             except OSError:
301                 pass
302
303         return valid_segments
304
305         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
306             futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
307             # Iterate in submission order to preserve temporal ordering
308             for future in futures:
309                 gemini_segments.extend(future.result())
310
311             try:
312                 os.rmdir(gemini_dir)
313             except OSError:
314                 pass
315
316             logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
317             return gemini_segments
318
319         def _populate_db(self, video_id: str, gemini_segments: List[dict], model_name: str)
-> List[Dict[str, Any]]:
320             """Phase 4: DB Population ? write confirmed AI segments, link them back to their
321             source candidate, and build the returned segments_data. Verbatim move."""
322             segments_data = []
323             for segment in gemini_segments:
324                 try:
325                     start = float(segment["start_time"])
326                     end = float(segment["end_time"])
327                     dur = end - start
328
329                     shot_count = segment.get("shuttle_exchange_count")
330                     if shot_count is None:
331                         shot_count = max(3, int(dur / 0.8))
332                     else:
333                         try:
334                             shot_count = int(shot_count)
335                         except (ValueError, TypeError):
336                             shot_count = max(3, int(dur / 0.8))
337
338                     metadata = {

```

```

339         "shot_count": shot_count,
340         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
341         "detector": f"yolo-hybrid-{model_name}"
342     }
343
344     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
345     if segment_id:
346         # Link back to the candidate that produced this segment. A candidate
347         # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
348         # the last one ? the link table records "this candidate was
confirmed".
349         candidate_id = segment.get("_candidate_id")
350         if candidate_id is not None:
351             self.db.link_candidate_to_segment(candidate_id, segment_id)
352         segments_data.append({
353             "id": segment_id,
354             "start_time": start,
355             "end_time": end,
356             "duration": dur,
357             "metadata": metadata
358         })
359     except Exception as e:
360         logger.warning(f"Skipping invalid segment struct: {e}")
361
362     return segments_data
363
364     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
365         self.person.lazy_load_model()
366
367         # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
368         provider = self._resolve_provider(video_path)
369         if provider is None:
370             return []
371
372         # Re-read the config values needed downstream (pure deterministic reads).
373         idx_cfg = self.config.get("indexing", {})
374         sport_profile = sport_registry.get(self.config.get("sport", "badminton"))
375         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
376         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
377
378         video_duration = get_video_duration(video_path)
379         if video_duration <= 0:
380             logger.error("Invalid video duration.")
381             return []
382
383         chunk_duration = idx_cfg.get("chunk_duration", 300)
384         chunk_overlap = idx_cfg.get("chunk_overlap", 30)
385         velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
386         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
387         frame_skip = idx_cfg.get("yolo_frame_skip", 3)
388         tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
389         handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
390         ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
391         log_candidates = idx_cfg.get("log_yolo_candidates", True)
392         store_candidates = idx_cfg.get("store_yolo_candidates", True)

```

```

393
394     # Snapshot of the params that produced these detections ? stored alongside each
395     # candidate so a fine-tuning run can reproduce / correlate against them.
396     detection_params = {
397         "velocity_thresh": velocity_thresh,
398         "merge_gap": merge_gap,
399         "frame_skip": frame_skip,
400         "tracker": tracker_config,
401         "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
402         "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
403         "min_action_window_duration": idx_cfg.get("min_action_window_duration",
404     2.0),
405     }
406
407     chunks_dir = os.path.join("output", "chunks_yolo")
408     os.makedirs(chunks_dir, exist_ok=True)
409
410     # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
411     all_candidate_windows = self._collect_candidate_windows(
412         video_id, video_path, video_duration, idx_cfg,
413         chunk_duration, chunk_overlap, velocity_thresh, merge_gap,
414         frame_skip, chunks_dir,
415     )
416
417     # Phase 2b: log + persist raw candidates (window index -> candidate_id).
418     candidate_ids = self._persist_candidates(
419         video_id, all_candidate_windows, detection_params, log_candidates,
420         store_candidates,
421     )
422
423     try:
424         os.rmdir(chunks_dir)
425     except Exception:
426         pass
427
428     # Phase 3: AI Provider Handoff
429     gemini_segments = self._run_ai_handoff(
430         video_id, video_path, video_duration, all_candidate_windows, candidate_ids,
431         provider, sport_profile, provider_name, chunk_duration, handoff_padding,
432         ai_max_workers,
433     )
434
435     # Phase 4: DB Population
436     return self._populate_db(video_id, gemini_segments, model_name)
437
438     # Register the segmenter
439     segmenter_registry.register(HybridYoloSegmenter)
440

```

9. assistant (2026-06-18T10:21:18.801Z)

Now let me compare the two `\_resolve\_provider` bodies carefully. They are NOT byte-identical:

```

**gemini.py** `_resolve_provider(self, model_name)` :
- Takes `model_name` as a parameter
- Returns `(sport_profile, provider)` tuple
- Uses `sport_profile = sport_registry.get(sport_name)` (captures result)
- Does NOT read `ai_model` (gets it from caller param)
- Does NOT log "Initializing..."

```

```

**yolo_hybrid.py** `_resolve_provider(self, video_path)` :
- Takes `video_path` as a parameter
- Returns `provider` only
- Calls `sport_registry.get(sport_name)` (discards result)
- Reads `model_name` internally from config

```

- Logs ``Initializing Hybrid Segmenter...`` at the end

The task says: "Only lift if the two bodies are byte-identical; if they diverge, report status='held'." These two bodies are clearly NOT byte-identical ? they differ in signature, return shape, model_name handling, and the trailing log line.

Let me look more closely to confirm the divergence is real and not something I'm misreading.

10. user (2026-06-18T10:21:21.764Z)

```
=== gemini _resolve_provider ===
def _resolve_provider(self, model_name: str):
    """Resolve the sport profile, provider, and API key (the process_video preamble).

    Returns (sport_profile, provider) on success, or None to signal any of the
    early-return cases (unknown sport, missing API key, unknown provider) ? the
    caller returns [] for None.
    """
    # Get sport profile config
    sport_name = self.config.get("sport", "badminton")
    try:
        sport_profile = sport_registry.get(sport_name)
    except KeyError as e:
        logger.error(str(e))
        return None

    # Get AI provider and model config
    idx_cfg = self.config.get("indexing", {})
    provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))

    # Get appropriate API key based on the provider
    api_key_env_var = f"{provider_name.upper()}_API_KEY"
    api_key = os.environ.get(api_key_env_var)
    if not api_key:
        logger.error(
            f"{api_key_env_var} environment variable is not set! "
            f"To run the {provider_name} segmenter, set your API key. "
            f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
        )
        return None

    try:
        provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
    except KeyError as e:
        logger.error(str(e))
        return None

    return sport_profile, provider

=== yolo_hybrid _resolve_provider ===
def _resolve_provider(self, video_path: str):
    """Phase 0: resolve sport profile + AI provider (with API key).

    Returns the configured provider instance, or ``None`` to signal that
    ``process_video`` should early-return ``[]`` (bad sport, missing API key,
    or unknown provider). Behaviour is identical to the inlined version.
    """
    # Get sport profile config
    sport_name = self.config.get("sport", "badminton")
    try:
        sport_registry.get(sport_name)
    except KeyError as e:
        logger.error(str(e))
        return None
```

```

# Get AI provider and model config
idx_cfg = self.config.get("indexing", {})
provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))

# Get appropriate API key based on the provider
api_key_env_var = f"{provider_name.upper()}_API_KEY"
api_key = os.environ.get(api_key_env_var)
if not api_key:
    logger.error(
        f"{api_key_env_var} environment variable is not set! "
        f"To run the {provider_name} segmenter, set your API key. "
        f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
    )
    return None

try:
    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
except KeyError as e:
    logger.error(str(e))
    return None

    logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
    return provider

```

11. assistant (2026-06-18T10:21:32.316Z)

This confirms my reading. The two `\_resolve\_provider` bodies are **NOT** byte-identical. There are material differences:

1. **Signature differs**: gemini takes `model\_name: str`; yolo_hybrid takes `video\_path: str`.
2. **`model\_name` source differs**: gemini receives `model\_name` as an argument; yolo_hybrid reads it internally via `model\_name = self.config.get("ai\_model", idx\_cfg.get("ai\_model", "gemini-2.5-flash"))`.
3. **Sport profile handling differs**: gemini captures `sport\_profile = sport\_registry.get(sport\_name)` and returns it; yolo_hybrid calls `sport\_registry.get(sport\_name)` and **discards** the result.
4. **Return shape differs**: gemini returns `(sport\_profile, provider)` tuple; yolo_hybrid returns `provider` only.
5. **Trailing side-effect differs**: yolo_hybrid emits `logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video: {video\_path}")`; gemini has no such log.

The task is explicit: "Only lift if the two bodies are byte-identical; if they diverge, report status='held'." The task description's premise (that the two have an "IDENTICAL private `\_resolve\_provider(self)` preamble") does not match the actual code on `origin/master` ? they diverge in signature, return type, internal vs. external `model\_name`, the discarded-vs-returned sport profile, and the trailing log line.

Lifting a single shared `\_resolve\_provider` to the base class would **require** changing behavior at the call sites (reconciling the differing return shapes, the `model\_name` source, and dropping/relocating the `logger.info` log string). That is a redesign with behavior changes (return types, log strings), not a pure move ? which the HARD RULES forbid forcing.

Per the discipline, I will STOP and report `status='held'`. Let me first run the gate suite to capture a clean baseline so the report is grounded, and verify nothing is mid-change.

12. user (2026-06-18T10:21:34.569Z)

=== diff vs origin/master ===

13. assistant (2026-06-18T10:21:39.452Z)

Working tree is clean (no changes made ? I have not edited production code, which is correct since I'm holding).

I have enough to report. The item cannot be done as a pure behavior-preserving move because the two `\_resolve\_provider` bodies on `origin/master` are not byte-identical ? the task's own gate ("Only lift if the two bodies are byte-identical; if they diverge, report status='held'") is triggered. I did not write characterization tests because the prerequisite for the refactor (identical bodies) fails, and the discipline says to STOP rather than force a behavior-changing redesign.

Let me record the precise diff between the two bodies as evidence for the report.

14. user (2026-06-18T10:21:41.596Z)

```
1,2c1,2
<     def _resolve_provider(self, model_name: str):
<         """Resolve the sport profile, provider, and API key (the process_video preamble).
<         ---
>     def _resolve_provider(self, video_path: str):
>         """Phase 0: resolve sport profile + AI provider (with API key).
4,6c4,6
<         Returns (sport_profile, provider) on success, or None to signal any of the
<         early-return cases (unknown sport, missing API key, unknown provider) ? the
<         caller returns [] for None.
<         ---
>         Returns the configured provider instance, or ``None`` to signal that
>         ``process_video`` should early-return ``[]`` (bad sport, missing API key,
>         or unknown provider). Behaviour is identical to the inlined version.
11c11
<         sport_profile = sport_registry.get(sport_name)
<         ---
>         sport_registry.get(sport_name)
18a19
>         model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
37c38,39
<         return sport_profile, provider
<         ---
>         logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
>         return provider
exit=1
```

15. assistant (2026-06-18T10:21:48.752Z)

The diff confirms the divergence concretely (exit=1, multiple substantive hunks). The two bodies differ in: signature (`model\_name` vs `video\_path`), docstring, sport-profile capture-vs-discard, internal `model\_name` derivation, the trailing `logger.info` side-effect, and return shape (`(sport\_profile, provider)` vs `provider`).

This is exactly the `held` condition the task specifies. I made no production edits and the working tree is clean. Reporting `status='held'`.

16. user (2026-06-18T10:22:05.152Z)

Structured output provided successfully

17. assistant (2026-06-18T10:22:13.578Z)

I've reported `status='held'`.

Summary: The item `base-resolve-provider-dedup` cannot be done as a behavior-preserving move. The task's binding precondition ? lift only if the two `\_resolve\_provider` bodies are byte-identical, otherwise hold ? fails on `origin/master`. A `diff` of `backend/pipeline/segmenters/gemini.py` (lines 66-102) against

``backend/pipeline/segmenters/yolo_hybrid.py`` (lines 57-95) returns `exit=1` with five substantive divergences:

1. Signature: ``model_name: str`` vs ``video_path: str``
2. `gemini` **captures and returns** ``sport_profile``; `yolo_hybrid` **discards** the ``sport_registry.get`` result
3. `yolo_hybrid` derives ``model_name`` internally from `config`; `gemini` takes it as an argument
4. `yolo_hybrid` emits a trailing ``logger.info("Initializing Hybrid Segmenter ...")`` side-effect; `gemini` does not
5. Return shape: ``(sport_profile, provider)`` tuple vs bare ``provider``

Lifting a single shared method would force changes to return types, the ``model_name`` source, and a log string at the call sites ? a redesign, not a pure move, which the HARD RULES forbid. I made no production edits; the worktree is clean and identical to ``origin/master``, so the four gates and base coverage (~80.9%) are unaffected. The task description's premise that these share an "IDENTICAL" preamble is no longer true ? they were independently refactored and diverged.